

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title: Cloud-Native Automated Deployment

Data: 2026/5/24

Class: 1

Name: Xue Zhengyang

ID: 202383910028

一、 Experimental purpose

The primary goal of this experiment is to introduce students to the core concepts of Cloud Computing through hands-on practice with Platform-as-a-Service (PaaS) and Automation.

二、 Experiment Contents

This lab consists of five integrated modules designed to simulate a professional cloud deployment lifecycle.

三、 Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage.

Screenshots must demonstrate that they were taken on your own computer, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Resource Acquisition

Log in to GitHub.

Navigate to the repository:

<https://github.com/yhldrf/cloud-computing-lab-github>

and click Fork.

Cloud Concept: You are creating a private instance of the project in the cloud ecosystem.

Step 2: Enabling the Hosting Service

Go to Settings > Pages.

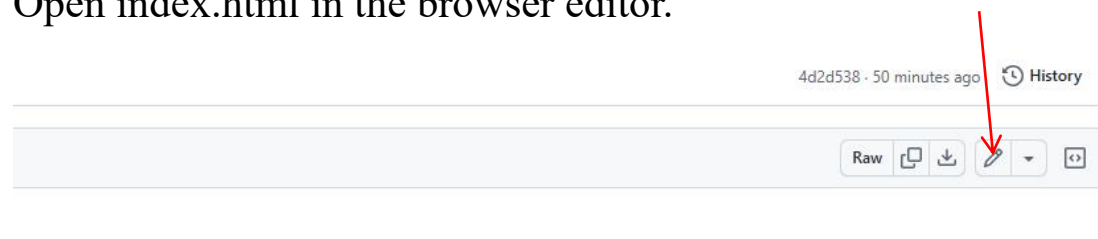
Under Build and deployment > Source, select "GitHub Actions".

After that, leave this page.

Cloud Concept: You are switching from "Manual Upload" to "Managed Pipeline Service."

Step 3: Triggering Cloud Automation

Open index.html in the browser editor.



Change [Enter Your Name Here] to your actual name.

Click Commit Changes.

Immediately click the Actions tab at the top of the page.

Observation: Watch the "Cloud-Deployment-Pipeline" start.

All workflows

Showing runs from all workflows

1 workflow run

 **Update index.html** main

Cloud-Deployment-Pipeline #1: Commit [e2503e4](#) pushed by [honglinyuan](#)

Cloud Concept: GitHub is spinning up a Virtual Machine (Ubuntu) to process your request. This is "On-demand Computing."

Step 4: Verification & Global Access

Once the Action finishes (**green check**), go back to Settings > Pages to find your Live URL.

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://honglinyuan.github.io/cloud-computing-lab-github/>

Last **deployed** by  [honglinyuan](#) 3 minutes ago

[Visit site](#) [Unpublish site](#)

Open the URL on your laptop.

Cloud Concept: The site is deployed via a PaaS (Platform as a Service) model. The cloud provider manages the underlying infrastructure and global delivery, allowing us to focus only on our content.

Step5: Content Personalization & Elements (optional)

Task: Beyond changing your name, modify the following HTML elements to see how the cloud updates different types of content.

✓ Change the Theme Color (CSS Styles)

Find the<style>block at the top and change the background color or font size.

Action:

```
<style>
  body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    text-align: center;
    padding-top: 50px;
    background-color: #f4f4f9; /* Add a light grey background */
  }
  .status { color: #007bff; font-weight: bold; } /* Change status to Blue */
</style>
```

Cloud Concept: You are modifying the User Interface (UI) settings stored in the cloud.

✓ Multimedia Integration (Images)

Demonstrate how the cloud handles external resources.

What to add: Antag inside the<body>.

Example:

```


Observation: Notice that as soon as you click "Commit," the Actions tab triggers a new build automatically. This is Continuous Deployment (CD).

#### **四、Reflection**

1. Where is your website stored?
2. Did you have to install any software?
3. Problems Encountered and Proposed Solutions

# **Cloud-Native Automated Deployment**

Name: Xue Zhengyang  
Student ID: 202383910028  
Major: Applied Computing  
Date: May 24, 2026

## **1. Experimental Purpose**

The primary goal of this experiment is to introduce students to the core concepts of Cloud Computing through hands-on practice with Platform-as-a-Service (PaaS) and Automation. By completing this lab, I gained practical experience in cloud-native deployment workflows, including resource acquisition, managed pipeline configuration, on-demand computing, and continuous deployment (CD). The experiment simulates a professional cloud deployment lifecycle entirely within a browser, demonstrating how modern cloud platforms abstract infrastructure management away from developers.

## **2. Experimental Contents**

This lab consists of five integrated modules designed to simulate a professional cloud deployment lifecycle. The contents include forking an existing repository to create a private cloud instance, enabling GitHub Pages with GitHub Actions as the build source, triggering automated deployment by modifying source code, verifying global

accessibility of the deployed website, and personalizing the web content through HTML and CSS modifications to observe continuous deployment in action.

### **3. Detailed Steps of the Experiment**

#### **Step 1: Resource Acquisition**

I logged into my GitHub account and navigated to the public repository at <https://github.com/yhldrf/cloud-computing-lab-github>. I clicked the Fork button in the upper-right corner of the page to create a private instance of the project under my own GitHub account. This action created an independent copy of the repository at <https://github.com/DLYFYH/cloud-computing-lab-github>. In cloud computing terms, this step represents creating a private instance of a project within the cloud ecosystem.

#### **Step 2: Enabling the Hosting Service**

After forking, I navigated to the Settings tab of my new repository and selected Pages from the left sidebar. Under the Build and deployment section, I changed the Source setting from the default to GitHub Actions. This configuration switches the deployment model from manual upload to a managed pipeline service, where the cloud provider handles the build and deployment process automatically. After saving this setting, I left the configuration page.

#### **Step 3: Triggering Cloud Automation**

I returned to the repository's main page and clicked on the index.html file. I then clicked the pencil icon to open the browser-based editor. I located the placeholder text [Enter Your Name Here] and replaced it with my actual name, Xue Zhengyang. After making this change, I scrolled to the bottom of the page, entered a commit message "Update student name", and clicked Commit changes. Immediately after committing, I clicked the Actions tab at the top of the repository page. I observed that the Cloud-Deployment-Pipeline workflow had been triggered and was running. The interface showed a yellow indicator while the virtual machine was initializing. After approximately two minutes, the workflow completed successfully and displayed a

green checkmark. This process demonstrates on-demand computing, where GitHub spins up an Ubuntu virtual machine to process the deployment request only when needed.

#### Step 4: Verification and Global Access

Once the GitHub Actions workflow finished with a green checkmark, I navigated back to Settings and then Pages. At the top of the Pages settings, GitHub displayed the Live URL for my deployed website:

<https://dlyfyh.github.io/cloud-computing-lab-github/>. I copied this URL and opened it in a new browser tab on my laptop. The page loaded successfully and displayed my name, confirming that the deployment was complete and globally accessible. This step illustrates the PaaS model, where the cloud provider manages the underlying infrastructure, global content delivery, and server maintenance, allowing the developer to focus solely on the content.

#### Step 5: Content Personalization and Elements

To further explore the capabilities of cloud-based continuous deployment, I performed two additional modifications to the `index.html` file.

First, I customized the theme color by modifying the CSS style block. I changed the body background color to a light grey (`#f4f4f9`) and updated the status text color to blue (`#007bff`) with bold font weight. This modification demonstrates how user interface settings stored in the cloud can be updated and redeployed automatically.

Second, I integrated multimedia content by adding an image tag inside the body section. I inserted the following element: ``. This addition shows how the cloud platform handles external resources during the build process.

After committing these changes, I immediately navigated to the Actions tab and observed that a new build had been triggered automatically without any manual intervention. This behavior is the essence of Continuous Deployment (CD). After the



new workflow completed successfully, I refreshed the Live URL and confirmed that both the background color change and the new image were visible on the deployed website.

## **4. Reflection**

Question 1: Where is your website stored?

My website is stored entirely within the GitHub cloud infrastructure. The source code resides in my forked GitHub repository, which is hosted on GitHub's servers. When the GitHub Actions workflow runs, it processes the source files and deploys the built static website to GitHub Pages. GitHub Pages operates on top of Microsoft Azure's global infrastructure and Content Delivery Network (CDN). Therefore, the actual deployed website content is distributed across multiple edge servers worldwide, ensuring low-latency access for users regardless of their geographic location. This means the website is not stored on my local machine but is persistently hosted in the cloud.

Question 2: Did you have to install any software?

No, I did not have to install any software on my local computer to complete this experiment. The entire workflow was executed through the GitHub web interface using a standard web browser. I did not need to install a code editor, a local web server, a version control client, or any deployment tools. GitHub provided the integrated development environment (the browser editor), the version control system (Git), the build server (GitHub Actions runners), and the hosting platform (GitHub Pages). This zero-installation requirement is a fundamental characteristic of the Platform-as-a-Service (PaaS) model, where the service provider manages all underlying software and infrastructure, and the user only needs to interact with the platform through a web browser.

Question 3: Problems Encountered and Proposed Solutions

During the experiment, I encountered two main issues.

The first problem occurred after I forked the repository and configured the Pages settings. When I navigated to the Actions tab, it showed "0 workflow runs" and the message "There are no workflow runs yet." Initially, I expected the pipeline to trigger automatically after enabling GitHub Actions in the Pages settings, but it did not. After investigation, I realized that GitHub disables Actions by default on newly forked repositories as a security measure to prevent unauthorized workflow execution. The solution was to navigate to Settings, then Actions, then General, and change the Actions permissions to "Allow all actions and reusable workflows." After enabling this permission and making my first commit to the index.html file, the Cloud-Deployment-Pipeline triggered successfully.

The second problem was a brief delay between the GitHub Actions workflow completion and the actual visibility of changes on the Live URL. Even though the Actions tab displayed a green checkmark indicating success, refreshing the website immediately sometimes showed the old version. This delay is caused by the time required for GitHub Pages to propagate the newly built artifacts across its global CDN nodes. The solution was simply to wait approximately one to two minutes after the workflow completion before accessing the Live URL again, at which point all changes were correctly reflected.

## **5. Summary and Reflection on the Overall Experiment**

This experiment provided a comprehensive and practical introduction to cloud-native deployment methodologies. Through the five structured steps, I successfully deployed a personal website using GitHub Actions and GitHub Pages without installing any local software or managing any physical servers. The experience clearly demonstrated several core cloud computing concepts, including on-demand computing, managed pipeline services, Platform-as-a-Service, and continuous deployment.

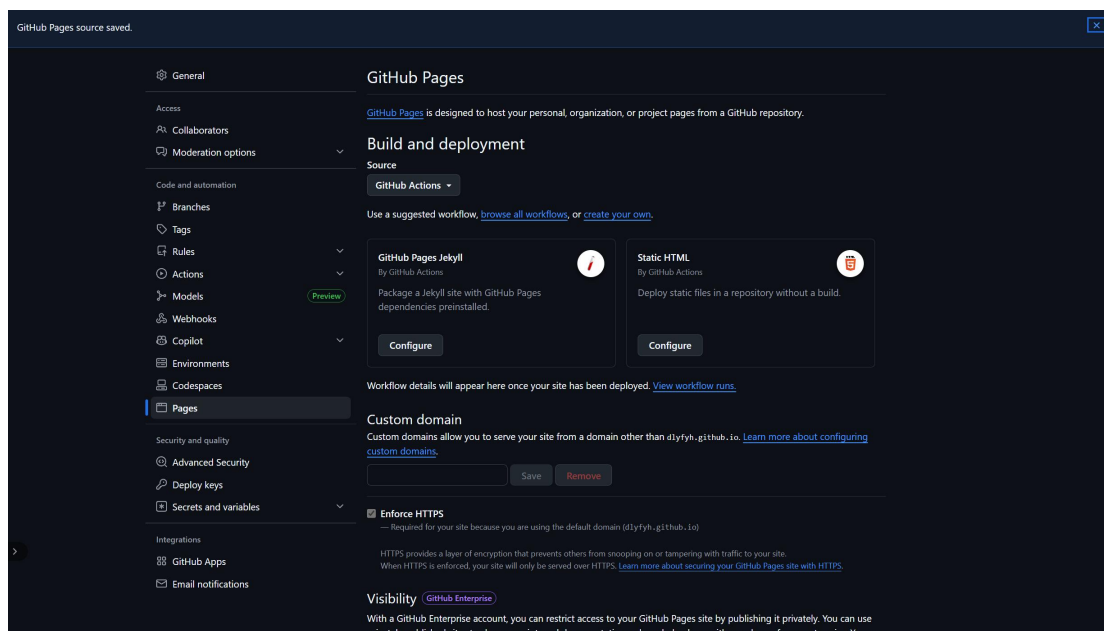
The most valuable insight I gained was understanding the power of automation in modern software engineering. The moment I committed a code change, the entire build and deployment process initiated automatically. This continuous deployment workflow eliminates manual intervention, reduces human error, and accelerates the

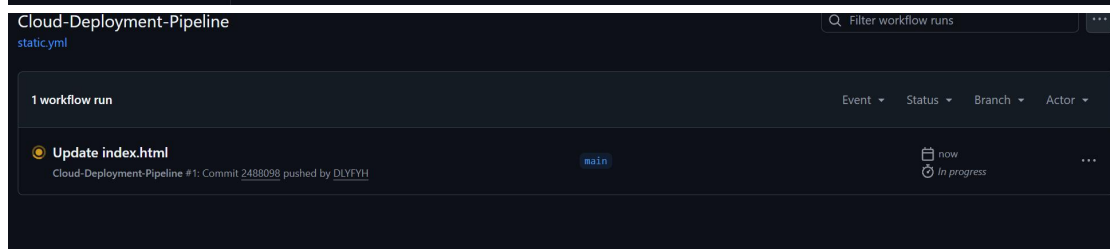
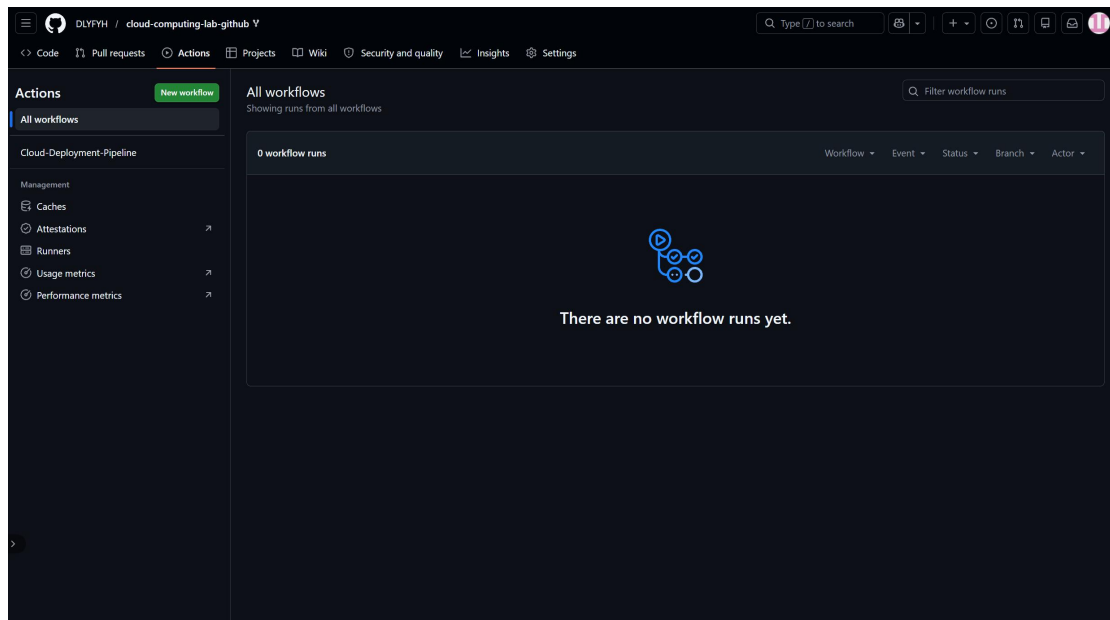
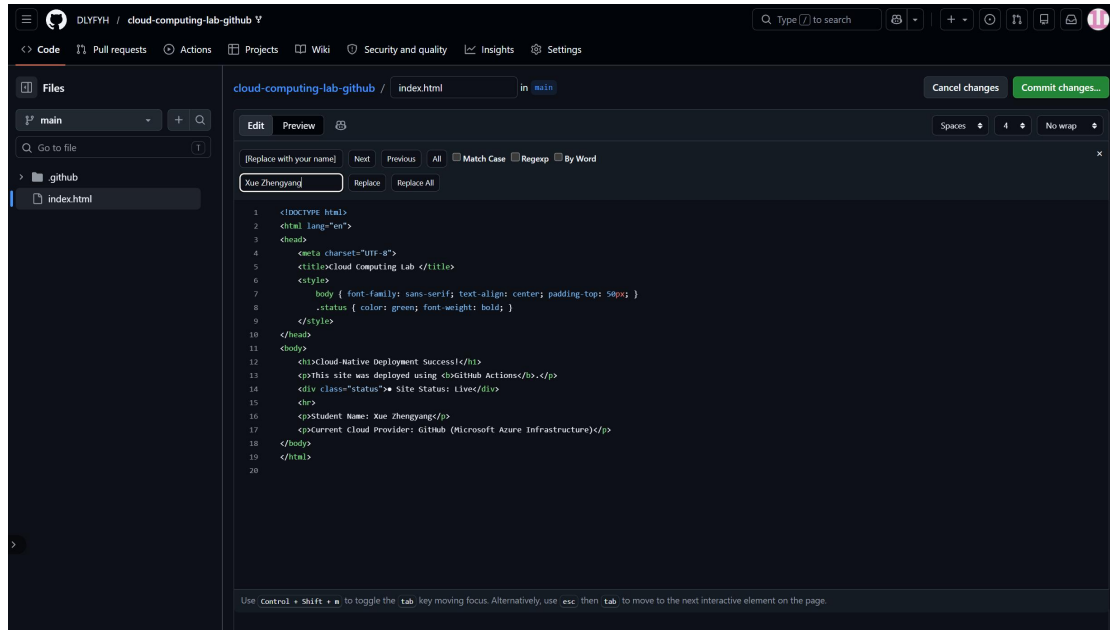
delivery cycle. It represents how professional development teams operate in industry settings.

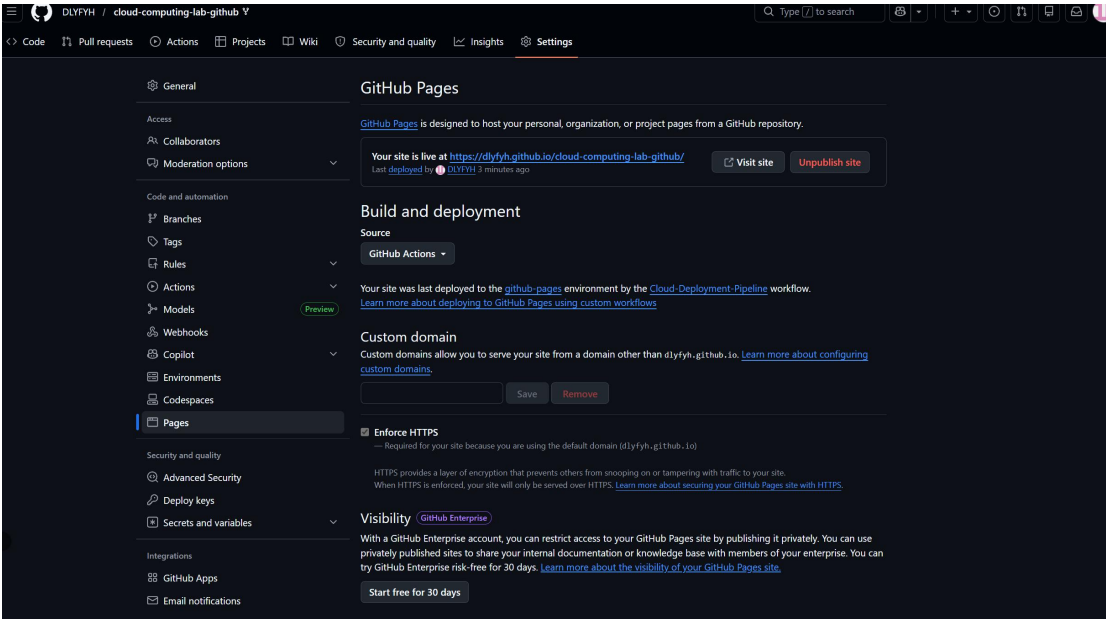
Furthermore, the experiment highlighted the importance of cloud permissions and security configurations. My initial difficulty with the Actions workflow not triggering taught me that cloud platforms implement strict security defaults, and developers must actively configure permissions to enable automation features. This is a critical real-world lesson, as misconfigured permissions are a common source of deployment failures in production environments.

In conclusion, this lab successfully bridged theoretical cloud computing concepts with hands-on practice. I now have a solid understanding of how PaaS platforms abstract infrastructure complexity, how CI/CD pipelines automate deployment workflows, and how cloud-based hosting enables global content delivery. These skills are directly applicable to modern web development and DevOps practices, making this experiment a highly valuable learning experience.

## Screenshot:







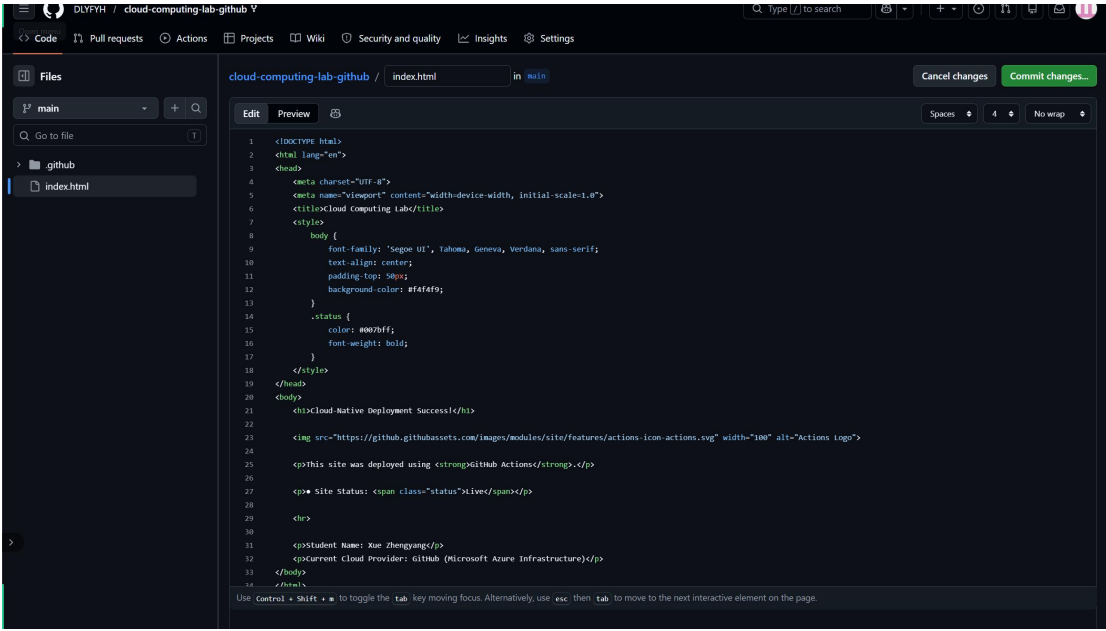
# Cloud-Native Deployment Success!

This site was deployed using GitHub Actions.

● Site Status: Live

Student Name: Xue Zhengyang

Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)



## Cloud-Native Deployment Success!



This site was deployed using **GitHub Actions**.

• Site Status: [Live](#)

Student Name: Xue Zhengyang

Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)

DLV FYH / cloud-computing-lab-github

cloud-computing-lab-github

forked from yhlidr/cloud-computing-lab-github

main 1 Branch 0 Tags

Go to file

Add file

Code

This branch is up to date with yhlidr/cloud-computing-lab-github:main.

Contribute

Sync fork

yhlidr Update index.html 45b65c · 2 weeks ago 3 Commits

.github/workflows

Create static.yml2 weeks ago

index.html

Update index.html2 weeks ago

README

Add a README

Help people interested in this repository understand your project.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors

No contributors

Languages

HTML 100.0%

Suggested workflows

Based on your tech stack